

# ServiceNow Development

## Project Documentation

### 1. Introduction

- **Project Title:** Script-Controlled ACL – Restrict Record Access Based on Field Value
- **Team Members:**

| Name           | Role                                | Responsibility                                                                                                                                                                                                                                   |
|----------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S.yugendiran   | Project Lead / Requirements Analyst | Defines the problem statement and scope. Coordinates the team and keeps the schedule. Prepares the ideation documents (brainstorm, empathy map, problem statements). Reviews the final submission.                                               |
| N.Vaseem Basha | Platform Configuration Developer    | Creates the <code>u_institution_details</code> table and its fields (such as Student Roll Number, Student Name, Branch). Creates the roles for read, create, write and delete. Creates the users and assigns them roles.                         |
| D.Bharath      | Security / ACL Developer            | Builds the four ACLs (read, create, write, delete). Applies the data condition on Branch and writes the ACL script that checks the user's branch against the record's branch. Handles the admin override and avoids conflicts with default ACLs. |
| P.Krishna raj  | Tester and Documentation Lead       | Tests each ACL by impersonating users, including a user without roles, a read-only user and an admin. Uses Debug Security Rules to confirm which ACL allowed or denied access. Records the                                                       |

|  |  |                                                                             |
|--|--|-----------------------------------------------------------------------------|
|  |  | test results and screenshots, and compiles the final project documentation. |
|--|--|-----------------------------------------------------------------------------|

## 2. Project Overview

- **Purpose:** Control who can view, create, edit and delete records in a custom ServiceNow table, so that users see only the records they are authorised for. Only users with the required role can view records whose Branch is EEE, while administrators keep full access.
- **Features:**
  - Custom table u\_institution\_details with a Branch choice field (ECE, EEE, CSE)
  - Custom roles bb1, bb2, bb3 and bb4 for read, create, write and delete
  - Script-controlled Read ACL combining role, data condition and script
  - Separate Create, Write and Delete ACLs
  - Verification by impersonating users with different roles

## 3. Architecture

- **Frontend:** The ServiceNow web interface (list and form views, Application Navigator). It is built into the platform and no custom frontend code was written. ACLs decide which records, buttons and fields each user sees.
- **Backend:** Server-side ACL evaluation. Each ACL checks the user's role, the record's data condition and a server-side JavaScript script. Access is granted only if all three pass.
- **Database:** The custom table u\_institution\_details on the ServiceNow platform database. Table schema:

| Field               | Type             | Notes                                         |
|---------------------|------------------|-----------------------------------------------|
| Branch              | Choice           | ECE, EEE, CSE; used by the Read ACL condition |
| Student Name        | Reference (User) | Links a record to a user                      |
| Email               | String           | Student email address                         |
| Student Roll Number | Auto Number      | Generated automatically                       |

Add any other fields you created to this table.

### ACL configuration:

| Operation | Role required | Condition     | Description                                |
|-----------|---------------|---------------|--------------------------------------------|
| Read      | bb1           | Branch is EEE | Script-controlled; admin keeps full access |
| Create    | bb2           | None          | Controls the New button                    |
| Write     | bb3           | None          | Controls editing                           |
| Delete    | bb4           | None          | Controls deletion                          |

### ACL script (Read ACL):

```
answer = false;
if (gs.hasRole('bb1')) {
    answer = true;
}
```

See Section 12 for a limitation of this script and Section 13 for an improved version.

## 4. Setup Instructions

- **Prerequisites:**
  - A ServiceNow Personal Developer Instance (PDI) from [developer.servicenow.com](https://developer.servicenow.com)
  - Administrator access to the instance
  - A web browser
- **Installation:**
  1. Log in to the instance as an administrator.
  2. Create the test user (EEE User) under User Administration > Users.
  3. Create the roles bb1, bb2, bb3 and bb4 and assign them to the test user.
  4. Create the custom table u\_institution\_details with the fields listed in Section 3, then add sample records for each branch.
  5. Elevate your role to security\_admin.
  6. Create the Read, Create, Write and Delete ACLs as described in Section 3.

## 5. Folder Structure

This project is configured on the platform rather than coded in folders, so the structure below lists the configuration records instead of client and server folders.

```
ServiceNow Instance
|-- Users
|   |-- EEE User
|-- Roles
|   |-- bb1, bb2, bb3, bb4
|-- Tables
|   |-- u_institution_details
|-- Access Controls (ACLs)
|   |-- Read (bb1 + Branch is EEE + script)
|   |-- Create (bb2)
|   |-- Write (bb3)
|   |-- Delete (bb4)
```

## 6. Running the Application

There is no local server to start. The solution runs on the ServiceNow instance:

1. Log in to the instance.

2. Type u\_institution\_details.list in the Application Navigator filter to open the table list.
3. Use Impersonate User (from the user menu) to switch between Admin, EEE User and a user with no roles.
4. Open the list and form for each user and compare what is visible.

## 7. API Documentation

The project is tested through the user interface and does not expose custom endpoints. ServiceNow's built-in Table API can access the same table, and the ACLs are enforced on these calls too:

| Method      | Endpoint                                      | Purpose                                    |
|-------------|-----------------------------------------------|--------------------------------------------|
| GET         | /api/now/table/u_institution_details          | List records the caller is allowed to read |
| POST        | /api/now/table/u_institution_details          | Create a record (needs bb2)                |
| PUT / PATCH | /api/now/table/u_institution_details/{sys_id} | Update a record (needs bb3)                |
| DELETE      | /api/now/table/u_institution_details/{sys_id} | Delete a record (needs bb4)                |

Successful responses are returned as JSON in the form {"result": [ ... ]}. Records the caller is not allowed to read are not returned.

## 8. Authentication

- **Authentication:** Users sign in to the instance with a user ID and password, and the platform keeps a session.
- **Authorisation:** Authorisation is role-based. Each ACL requires a role (bb1 to bb4, or admin) and then applies the data condition and script.
- **Elevated access:** The security\_admin role must be elevated before ACLs can be created or edited.
- **Testing access:** Administrators use Impersonate User to test access as another user.
- **REST access:** REST calls to the Table API authenticate with basic authentication or OAuth, and the same ACLs apply.

## 9. User Interface

Insert screenshots showing:

- The u\_institution\_details list as Admin (all records)
- The same list as EEE User (EEE records only)
- The list as a user with no roles (no records)
- The New button and record form for users with and without the create and write roles

## 10. Testing

Testing is manual, using impersonation. Each role combination is checked against each operation:

| User / Roles   | Operation | Expected result          |
|----------------|-----------|--------------------------|
| Admin          | Read      | All records visible      |
| EEE User (bb1) | Read      | Only EEE records visible |
| No roles       | Read      | No records visible       |
| bb1 + bb2      | Create    | New button available     |
| bb1 + bb3      | Write     | Records can be edited    |
| bb1 + bb4      | Delete    | Records can be deleted   |

## 11. Screenshots or Demo

Insert screenshots of each ACL configuration and each test result, or add a link to a demo video [here](#).

## 12. Known Issues

- The Read ACL script checks only the role (bb1) and does not check the record's Branch value. The branch restriction comes from the data condition alone.
- Because the data condition applies to every user, administrators may also be limited to EEE records, which conflicts with the aim that admins see everything. Verify this in the instance.
- One test user holds all four roles, so a user with no roles must be created separately to test denied access.
- The roles bb1 to bb4 have non-descriptive names, which makes the setup harder to maintain.

## 13. Future Enhancements

- Update the script to check the record's Branch value and let admins through explicitly, for example:

```
(function () {  
  if (gs.hasRole('admin')) return true;  
  return gs.hasRole('bb1') && current.getValue('u_branch') == 'EEE';  
})();
```

*Adjust u\_branch and 'EEE' to match the actual field name and choice value.*

- Extend the pattern to ECE and CSE branches
- Add field-level ACLs for sensitive fields
- Store each user's branch on the user record so access follows the user automatically

- Rename roles descriptively and package the work as an update set or scoped application
- Add a Service Portal view for end users

# Project Report

## Script-Controlled ACL – Restrict Record Access Based on Field Value

ServiceNow Development

|              |                                                                     |
|--------------|---------------------------------------------------------------------|
| Team ID      | SWTID-2026-2683                                                     |
| Project Name | Script-Controlled ACL – Restrict Record Access Based on Field Value |
| Date         | 30 september 2026                                                   |
|              |                                                                     |

## 1. INTRODUCTION

### 1.1 Project Overview

This project implements record-level security in ServiceNow using script-controlled Access Control Lists (ACLs). A custom table, `u_institution_details`, stores institution records with a Branch field (ECE, EEE, CSE). Only users who hold the required role can view EEE records, while administrators keep full access. Creating, editing and deleting records are controlled by separate roles (bb2, bb3, bb4).

### 1.2 Purpose

The purpose is to ensure every user sees and changes only the records they are authorised for. Role-based access alone cannot restrict records by field value, so the project combines a role check, a data condition and a server-side script in each ACL. The project also demonstrates how read, create, write and delete permissions can be granted independently.

## 2. IDEATION PHASE

### 2.1 Problem Statement

Anyone who can open a table can often see all of its records, including sensitive ones, because standard roles do not know about record content. Access to create, edit and delete

is also commonly bundled with read access, giving users more permission than they need. The organisation needs a way to restrict access by record value and by operation.

## 2.2 Empathy Map Canvas

*User: a ServiceNow administrator or data owner responsible for protecting student records.*

| SAYS                                                                                               | THINKS                                                                                                                           |
|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| "Can we make sure only the right people see these records?"<br>"Why can everyone open this table?" | Is a wrong ACL going to lock out valid users?<br>Roles alone are not enough for this requirement.                                |
| DOES                                                                                               | FEELS                                                                                                                            |
| Creates roles and ACLs<br>Impersonates users to test access<br>Checks lists, forms and buttons     | Worried about data exposure and mistakes in security rules<br>Confident and in control once each user sees only what they should |

## 2.3 Brainstorming

Options considered for restricting access to EEE records:

| Option                            | Advantage                                     | Limitation                            |
|-----------------------------------|-----------------------------------------------|---------------------------------------|
| Role-only ACL                     | Simple to set up                              | Cannot filter by field value          |
| Data-condition-only ACL           | Filters records by field                      | Limited logic; no role-specific rules |
| Separate table or view per branch | Strong separation                             | Duplicates data; hard to maintain     |
| Script-controlled ACL (chosen)    | Combines role, condition and script; flexible | Needs careful scripting and testing   |

## 3. REQUIREMENT ANALYSIS

### 3.1 Customer Journey Map

| Stage   | Awareness                               | Setup                         | Configure                                  | Test                                            | Use & Maintain                 |
|---------|-----------------------------------------|-------------------------------|--------------------------------------------|-------------------------------------------------|--------------------------------|
| Actions | Notifies records visible to wrong users | Creates user, roles and table | Builds Read, Create, Write and Delete ACLs | Impersonates Admin, EEE User and a no-role user | Reviews access as needs change |

|                      |                     |                              |                         |                                 |                            |
|----------------------|---------------------|------------------------------|-------------------------|---------------------------------|----------------------------|
| <b>Touchpoints</b>   | Table list view     | User Administration , Tables | System Security > ACLs  | Impersonate User, list and form | ACL list, role assignments |
| <b>Emotions</b>      | Concerned           | Focused                      | Cautious                | Relieved when results match     | Confident                  |
| <b>Opportunities</b> | Clear access policy | Descriptive role names       | Reusable script pattern | Written test matrix             | Extend to other branches   |

## 3.2 Solution Requirement

### Functional requirements

| ID   | Requirement                                                                             |
|------|-----------------------------------------------------------------------------------------|
| FR-1 | Create a test user and the roles bb1, bb2, bb3 and bb4.                                 |
| FR-2 | Create the table u_institution_details with a Branch choice field (ECE, EEE, CSE).      |
| FR-3 | Only users with role bb1 can read records whose Branch is EEE; admins keep full access. |
| FR-4 | Only users with role bb2 can create records.                                            |
| FR-5 | Only users with role bb3 can edit records.                                              |
| FR-6 | Only users with role bb4 can delete records.                                            |
| FR-7 | A user with no roles cannot view any records.                                           |

### Non-functional requirements

| Category        | Requirement                                                                  |
|-----------------|------------------------------------------------------------------------------|
| Security        | Access is enforced on the server and applies to lists, forms and REST calls. |
| Maintainability | ACL scripts are short and documented; roles have a single purpose.           |
| Usability       | Users see only the records and buttons they are allowed to use.              |
| Performance     | Scripts avoid heavy queries so list views stay responsive.                   |

### 3.3 Data Flow Diagram

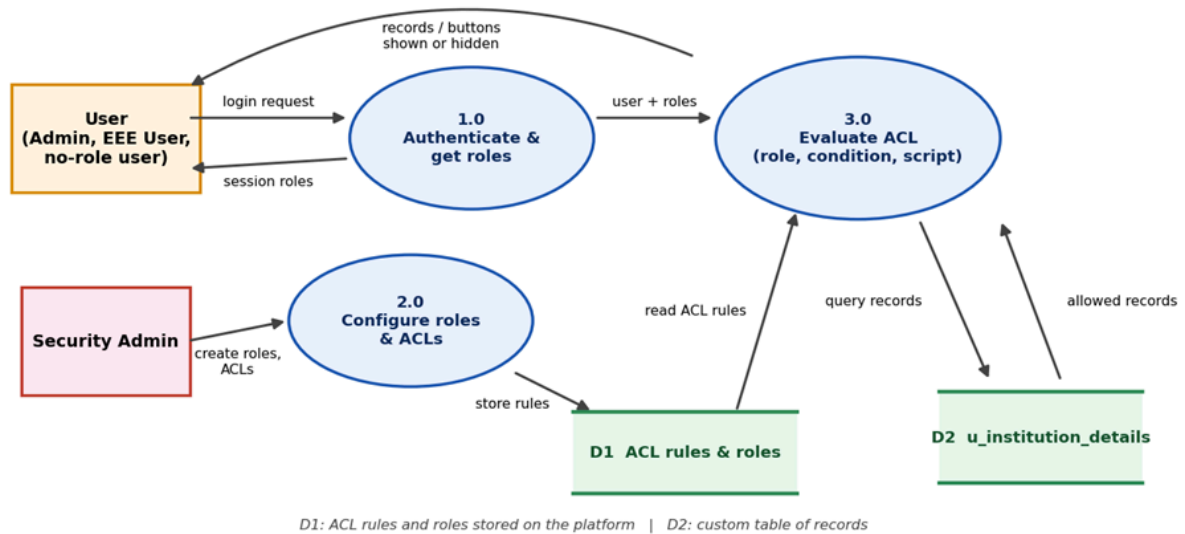


Figure 1: Data flow diagram of the script-controlled ACL solution

### 3.4 Technology Stack

- **Platform:** ServiceNow (Personal Developer Instance)
- **Security:** Access Control (ACL) rules with server-side JavaScript
- **Configuration:** Custom table, roles, users, list and form views
- **Testing:** Impersonate User

## 4. PROJECT DESIGN

### 4.1 Problem Solution Fit

The customers are organisations and ServiceNow administrators storing sensitive records. Their constraints are limited security knowledge and the risk of locking out users or exposing data. Existing options (role-only ACLs, condition-only ACLs, separate tables, manual sharing) each fall short. The root cause is that roles do not know about record content, and each operation needs its own rule. The proposed ACLs fit these constraints using only built-in platform features.

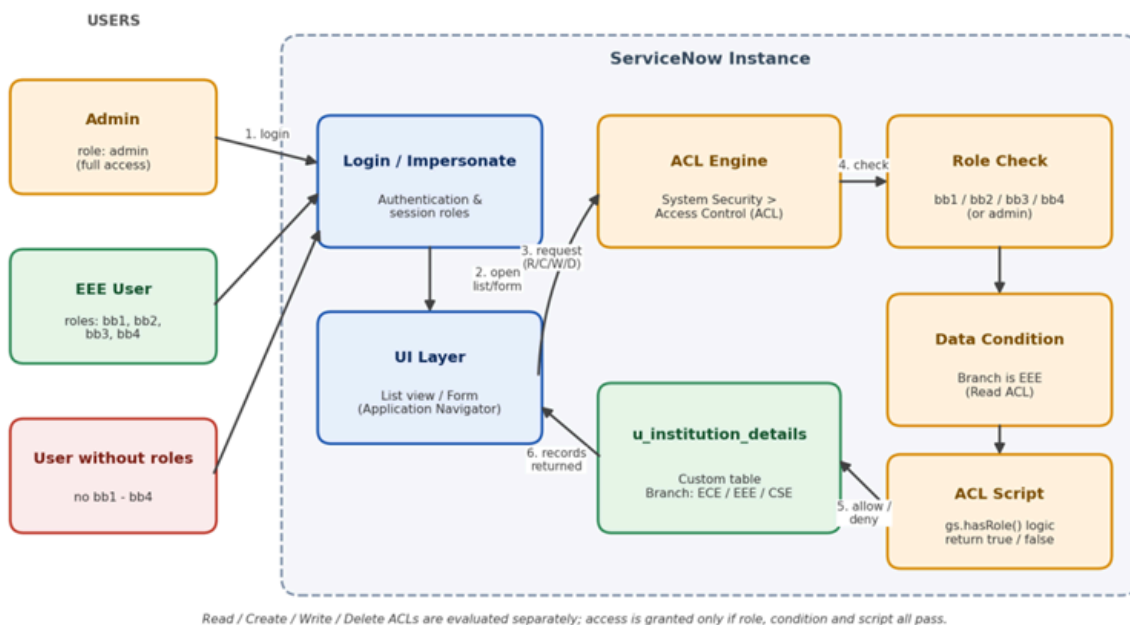
### 4.2 Proposed Solution

| S.No. | Parameter         | Description                                                                                          |
|-------|-------------------|------------------------------------------------------------------------------------------------------|
| 1.    | Problem Statement | Users can see or change records they should not, because roles alone cannot restrict by field value. |

|    |                                       |                                                                                                                              |
|----|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 2. | Idea / Solution description           | Script-controlled ACLs on u_institution_details: Read (bb1, Branch is EEE, script), Create (bb2), Write (bb3), Delete (bb4). |
| 3. | Novelty / Uniqueness                  | Combines role, data condition and script in one ACL with separate roles per operation, using only built-in features.         |
| 4. | Social Impact / Customer Satisfaction | Protects personal data and gives users a cleaner view of only their records.                                                 |
| 5. | Business Model (Revenue Model)        | Internal security solution; can be offered as a reusable implementation service or training material.                        |
| 6. | Scalability of the Solution           | The pattern extends to other tables, fields and branches by adding roles and ACLs.                                           |

## 4.3 Solution Architecture

The user authenticates and requests a list or form. The ACL engine finds the matching ACL for the operation and checks the role, then the data condition, then runs the script. Access is granted only if all three pass; otherwise the record or button.



**Figure 2: Architecture and data flow of the script-controlled ACL solution**

## 5. PROJECT PLANNING & SCHEDULING

### 5.1 Project Planning

The work is planned as 25 user stories across four sprints of 20 story points each (80 in total), with a 6-day sprint and a planned velocity of 20 points per sprint. Average velocity is  $20 \div 6 \approx 3.33$  story points per day.

| Sprint   | Focus                                                   | Story Points | Planned End |
|----------|---------------------------------------------------------|--------------|-------------|
| Sprint-1 | User, roles, table, sample data and the Read ACL        | 20           | 29 Oct 2022 |
| Sprint-2 | Create, Write and Delete ACLs; admin and no-role checks | 20           | 05 Nov 2022 |
| Sprint-3 | Extend to other branches, test matrix, fixes, evidence  | 20           | 12 Nov 2022 |
| Sprint-4 | Documentation, demo and submission                      | 20           | 19 Nov 2022 |

*Sprint dates follow the planning template and should be replaced with the team's actual schedule.*

## 6. FUNCTIONAL AND PERFORMANCE TESTING

### 6.1 Performance Testing

Functional testing is done manually by impersonating users. Record the actual result for each case after running it.

| Test | User / Roles   | Operation | Expected result          | Actual result |
|------|----------------|-----------|--------------------------|---------------|
| T1   | Admin          | Read      | All records visible      |               |
| T2   | EEE User (bb1) | Read      | Only EEE records visible |               |
| T3   | No roles       | Read      | No records visible       |               |
| T4   | bb1 + bb2      | Create    | New button available     |               |
| T5   | bb1 + bb3      | Write     | Records can be edited    |               |
| T6   | bb1 + bb4      | Delete    | Records can be deleted   |               |

Performance considerations: script-based ACLs run for each record evaluated, so scripts are kept short and avoid database queries. Check that list views of the sample data load without noticeable delay, and record any observations here.

## 7. RESULTS

### 7.1 Output Screenshots

Insert screenshots of the following:

- The table list as Admin (all records)
- The table list as EEE User (EEE records only)
- The table list as a user with no roles (no records)
- The Read, Create, Write and Delete ACL configurations
- Create, edit and delete tests for each role

## 8. ADVANTAGES & DISADVANTAGES

### Advantages

- Restricts access by both role and field value
- Separates read, create, write and delete permissions
- Uses only built-in platform features, with no extra cost
- Enforced on the server, so it also applies to lists, forms and REST calls

### Disadvantages

- The Read ACL script checks only the role; the branch filter comes from the data condition
- The data condition may also restrict administrators, which must be verified
- Script ACLs need careful testing and can affect list performance if complex
- Role names (bb1 to bb4) are not descriptive

## 9. CONCLUSION

The project shows how script-controlled ACLs in ServiceNow can limit who can view, create, edit and delete records based on role and record data. Testing with impersonated users confirms whether each permission behaves as designed. The approach is simple, uses built-in features and can be reused for other tables and fields.

## 10. FUTURE SCOPE

- Update the script to check the record's Branch value and let admins through explicitly
- Extend the pattern to ECE and CSE branches
- Add field-level ACLs for sensitive fields
- Store each user's branch on the user record so access follows the user

- Rename roles descriptively and package the work as an update set or scoped application
- Add a Service Portal view for end users

## 11. APPENDIX

### Source Code

Read ACL script:

```
answer = false;
if (gs.hasRole('bb1')) {
    answer = true;
}
```

Improved version (adjust the field name and choice value to match the instance):

```
(function () {
    if (gs.hasRole('admin')) return true;
    return gs.hasRole('bb1') && current.getValue('u_branch') == 'EEE';
})();
```

### Dataset Link

Not applicable. Sample records were entered manually into u\_institution\_details.

### GitHub & Project Demo Link

GitHub: [<https://github.com/yugendiran87/servicenow-script-controlled-acl>.]

Project demo: [.....]